#show table.cell: set par(justify: false) #show table.cell: set text(size: 9pt, hyphenate: false)

ФСП Таланты

Платформа подбора ИТ-специалистов с подтверждённой категорией. Сопроводительная документация

Содержание

ФСП Таланты — платформа подбора ИТ-специалистов с подтверждённой категорией 2
Коротко о решении 2
1. Функциональная архитектура 3
2. Компонентная архитектура 3
3. Справочники 4
4. Механика тестирования 4
5. Механика подбора и категоризация 6
6. Процедура валидации и результаты 7
7. Интеграция с ФСП 9
8. Безопасность и 152-ФЗ 10
9. API 10
10. Вне MVP: концепции 11
11. Библиотеки и компоненты 11
12. Сборка, развёртывание и запуск 12

ФСП Таланты — платформа подбора ИТ-специалистов с подтверждённой категорией

Хакатон «Лидеры цифровой трансформации 2026», спецтрек Федерации спортивного программирования России.

Репозиторий: https://github.com/K1NGSM4N3/fsp-hack · Swagger UI развёрнутого решения: `/api/docs` · Отчёты валидации: `docs/validation/`

Коротко о решении

Работодатель не ждёт откликов, а сам выходит на кандидата. Кандидаты разложены по **категориям** — специализация × грейд, — и категорию даёт только **адаптивный тест**, а не резюме. Внутри категории выше стоят те, у кого сильнее результат теста, подтверждены навыки и есть достижения ФСП. Работодатель описывает потребность, получает **объяснённую подборку** и отправляет **приглашение с обязательной зарплатной вилкой**. Кандидат видит условия до начала общения, а его контакты открываются работодателю только после согласия.

Что отличает решение:

1. **Тест, который бесполезно сливать.** Задания — параметрические шаблоны: у каждого кандидата свои числа, код и ответы, а ответ вычисляется генератором. Сложность сопоставима, потому что шаблоны откалиброваны по IRT и все кандидаты измеряются на одной шкале. В симуляции утечки у фиксированного теста при 50 % слитых ответов проходят 99,9 % слабых кандидатов, у нашего — 9–10 %, как и без утечки (§6.2).
2. **Механика контакта «работодатель ⟶ кандидат».** Анонимная карточка, объяснение попадания, вилка в первом сообщении, квота приглашений против спама, причины отказов обратно работодателю, статистика спроса кандидату (§5).
3. **Измеримое качество.** Собственная процедура валидации на синтетике со скрытой «истиной»: доля релевантных в топ-10 — 0,94 против 0,43 у поиска по ключевым словам резюме; итоговый грейд совпадает с истинным в 87 % случаев, при пересдаче — κ = 0,74 (§6).
4. **ФСП ID на Keycloak уже сейчас.** Привязка через OIDC + PKCE к заглушке ФСП ID на настоящем Keycloak 26, достижения из реестра влияют на ранжирование, кандидат без истории ФСП не штрафуется (§7).

1. Функциональная архитектура

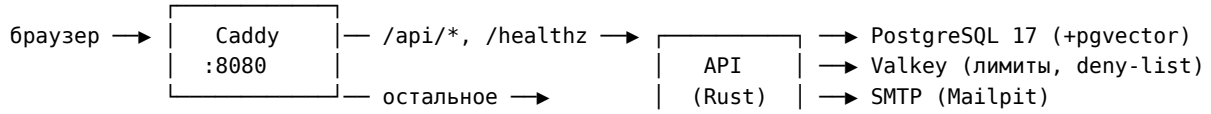
1.1. Роли и сквозные сценарии

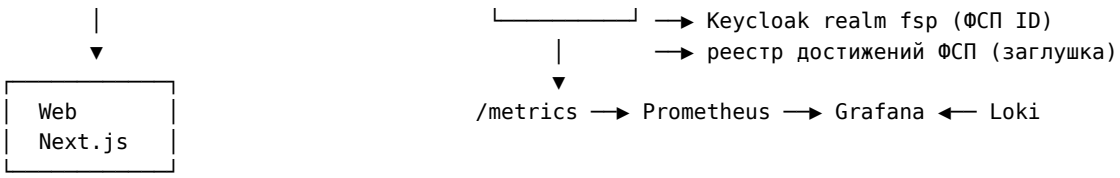
Роль	Сценарий
Кандидат	Регистрация по email с подтверждением и согласиями ⟶ профиль (форма или PDF-резюме с распознаванием) ⟶ опрос (направление, специализация, стек, заявленный грейд) ⟶ адаптивный тест ⟶ категория ⟶ входящие приглашения с вилкой ⟶ принять / отклонить с причиной. Дополнительно: вакансии и отклики, регулярные задания, привязка ФСП ID, PDF-профиль, настройки приватности
Работодатель	Профиль компании ⟶ описание потребности (свободным текстом, система распознаёт специализацию, грейд, стек и вилку) ⟶ рекомендованные категории ⟶ подборка с объяснениями, уточнение фильтров без потери ⟶ анонимный профиль ⟶ приглашение с вилкой ⟶ контакты после принятия. Дополнительно: поиск по всему банку, вакансии и отклики, регулярные задания, статистика приглашений

1.2. Покрытие требований ТЗ

Требование	Реализация
Категория = специализация × грейд по опросу и тесту, не по резюме	6 специализаций × 3 грейда; категорию присваивает только сданный тест (§4, §5.3)
Механика тестирования, устойчивая к утечкам, с сопоставимой сложностью	62 параметрических шаблона, IRT 3PL, адаптивный тест (§4)
Грейд не понижается принудительно; пересдача ниже / выше; ограничение по времени	Политика грейда: шаг вниз сразу после провала, дотест вверх после уверенной сдачи, смена грейда не чаще раза в 90 дней (§4.5)
Ранжирование внутри категории по тесту и ФСП	Сила профиля = 0,6 · тест + 0,25 · ФСП + 0,15 · актуальность (§5.3)
Работодатель находит категорию и выходит на кандидата с предложением и зарплатой	Рекомендованные категории, подборка, поиск по банку, приглашение без привязки к вакансии (§5)
Объяснимость выдачи; уточнение без потери подборки	Разложение балла по признакам, причины и пробелы; подборка хранится, закреплённые кандидаты переживают уточнение (§5.4)
Зарплата «от–до» обязательна; контакты после принятия / отклика	Проверяется в API и ограничениях БД; контакты недоступны на уровне типов (§8.3)
Статусы приглашений и откликов	отправлено ⟶ просмотрено ⟶ принято / отклонено; истекло, отозвано
Связь с ФСП ID; корректная обработка без истории ФСП	OIDC + PKCE к Keycloak, реестр достижений, нейтральное «Нет данных ФСП» (§7)
PDF-резюме с распознаванием, стандартизированный PDF-профиль	Извлечение текста + словари и правила; PDF в фирменном шрифте ФСП
Регулярные задания, вакансии и отклики (желательно)	Реализованы
Защита от фиктивных вакансий, ATS (концепция)	Квоты, жалобы, концепция антифейка и интеграции (§10)

2. Компонентная архитектура





Backend — модульный монолит на Rust. Логика вынесена в чистые крейты без ввода-вывода и часов, поэтому одна и та же реализация работает в API и в симуляторе валидации:

Крейт	Назначение
`catalog`	Справочники: направления, специализации, грейды и их пороги на шкале θ, категории, отрасли
`assessment`	Тестирование: модель IRT, адаптивный выбор заданий, 62 шаблона, политика смены грейда
`matching`	Подбор: словарь навыков с синонимами и графом близости, скоринг с объяснениями, ФСП-скор, разбор текста вакансии и резюме
`db`	Типизированные SQL-запросы, сгенерированные clorinde из `db/queries/*.sql`
`api`	HTTP API (axum), аутентификация, OpenAPI, сервисы
`fsp-mock`	Заглушка реестра достижений ФСП
`validate`	Процедура валидации, формирует `docs/validation/`

Frontend — Next.js (App Router, TypeScript) с клиентом, сгенерированным из OpenAPI. Права проверяет только API.

Данные: PostgreSQL — пользователи (email и ФИО зашифрованы), профили, навыки с источником (заявлен / из резюме / подтверждён тестом / ФСП), сессии тестов (хранятся только ссылки на экземпляры заданий), история грейдов, потребности и сохранённые подборки, приглашения и их история, журнал доступа к контактам, согласия.

Развёртывание: docker compose с профилями — ядро (Caddy, web, API, PostgreSQL, Valkey, Mailpit), `fsp` (Keycloak и реестр), `obs` (Prometheus, Loki, Alloy, Grafana), `full` — всё. API в работе занимает несколько мегабайт памяти, весь стек укладывается в недорогой VPS.

3. Справочники

- **Направления и специализации:** разработка (бэкенд, фронтенд), инфраструктура (DevOps/SRE), качество (QA), данные и ИИ (ML/DS), безопасность.
- **Грейды:** Junior, Middle, Senior. Пороги на шкале способности θ: −1, 0, +1.
- **Навыки:** 83 навыка, 491 синоним на русском и английском, включая сленг («постгрес», «кубер», «джава»), 77 рёбер близости (PostgreSQL ~ MySQL, React ~ Vue…).

4. Механика тестирования

4.1. Идея

Единый набор заданий утекает; генерация нейросетью «под каждого» не гарантирует сопоставимую сложность. Мы разделили две задачи:

- **Неповторимость** даёт генератор: шаблон задания по случайному `seed` выдаёт новый экземпляр — другие числа, код, имена, порядок и набор вариантов. Правильный ответ **вычисляется** генератором (мини-симулятор event loop JavaScript, расчёт подсети, выполнение SQL на сгенерированной таблице и т. п.), а не хранится.
- **Сопоставимость** даёт калибровка: у каждого уровня шаблона есть параметры модели IRT — дискриминативность `a`, сложность `b`, угадывание `c`. Способность всех кандидатов измеряется на общей шкале θ, поэтому разные задания дают сравнимый результат.

Сложность внутри шаблона задают «радикалы» (например, число асинхронных задач в event loop), а «инциденталы» (числа, имена, порядок) меняются без влияния на сложность.

4.2. Банк заданий

62 шаблона: 52 вычислительных генератора и 10 вариативных пулов (5–7 формулировок, 3–4 равноправных правильных ответа и 7–10 дистракторов на каждый вопрос). У каждого шаблона 1–3 уровня сложности, b ≈ −1,4… +1,4. Каждый уровень каждого шаблона даёт не менее 1961 различимого экземпляра на 2000 случайных `seed`; это закреплено тестом.

Специализация	Шаблонов	Тем	Примеры
Бэкенд	24	10	изменяемый аргумент по умолчанию в Python, порядок `defer` в Go, выбор индекса, token bucket, LRU, аномалии изоляции, SQL `GROUP BY / HAVING`
Фронтенд	15	6	порядок вывода в event loop, замыкания в цикле, приведение типов, специфичность CSS, box model, батчинг состояния React
DevOps	18	10	кэш слоёв Docker, ресурсы Kubernetes, CIDR, `location` в nginx, cron, права Linux, `increase()` в Prometheus, бюджет ошибок
QA	18	8	граничные значения, классы эквивалентности, pairwise, диаграммы состояний, таблицы решений, проверки API
ML	18	8	метрики по матрице ошибок, теорема Байеса, градиентный шаг, k-means, `groupby` в pandas, формы массивов NumPy
Безопасность	15	8	класс уязвимости во фрагменте кода, энтропия пароля, XOR-шифр, Base64, обход фильтров, правила межсетевого экрана

Типы ответа: выбор одного, выбор нескольких, число, текст. Задания и интерфейс — на русском, код — на соответствующем языке.

4.3. Адаптивный тест

Параметр	Значение
Модель	3PL; $c = 1/k$ для выбора из k вариантов, 0 для числового и текстового ответа
Оценка θ	Апостериорное среднее (EAP), сетка из 81 точки на [−4; 4], априорное N(порог заявленного грейда; 1)
Выбор задания	Максимум информации Фишера в текущей θ, случайный выбор среди пяти лучших (контроль экспозиции), квоты по темам, без повторов шаблона
Остановка	Не менее 8 и не более 20 заданий; стоп, когда 95-процентный интервал по θ не содержит порог заявленного грейда
Решение	$\theta \geqslant$ порога $\longrightarrow$ тест сдан
«Уверенно сдал»	Измеренный грейд выше заявленного или $P(\theta \geqslant$ следующего порога$) \geqslant 0{,}9 \longrightarrow$ предлагается дотест на грейд выше
Время	Лимит на задание (60–180 с), просроченный ответ засчитывается как неверный
Аномалии	Статистика person-fit l_z; при $l_z < -2$ («верно на трудных, неверно на лёгких») результат помечается для проверки

Стек кандидата из опроса повышает приоритет шаблонов с совпадающими навыками, но шкала и пороги остаются общими. Навыки из правильно решённых заданий получают отметку «подтверждён тестом».

4.4. Почему тест устойчив к распространению заданий

Что утекло	Что даёт утечка
Конкретный экземпляр с ответом	Практически ничего: следующий кандидат получит другой `seed`, а значит другой код и другой ответ
Шаблон («тут спрашивают порядок вывода event loop»)	Понимание, как решать такой класс задач, — ровно тот навык, который мы измеряем
Весь банк шаблонов	Ответы всё равно нужно вычислить, а на каждое задание действует лимит времени

Количественная проверка — §6.2: даже при базе из ~42 тыс. слитых экземпляров совпадения встречаются в 0,02–0,05 % заданий, и доля прошедших слабых кандидатов не растёт (9–10 %), тогда как у фиксированного теста — до 100 %.

Роль нейросетей: только офлайн — предложить новый шаблон, который проходит ревью и пробную калибровку (показывается кандидатам без влияния на оценку) до попадания в рабочий банк. Сопоставимость обеспечивает калибровка, а не генерация.

4.5. Правила смены грейда

- **Первичное размещение.** Кандидат выбирает грейд и проходит тест. Не сдал — сразу может пройти тест на грейд ниже; повтор того же или более высокого грейда — через 30 дней (иначе тест можно было бы перебирать). Не сдал Junior — пересдача через 30 дней, категории пока нет.
- **Уверенно сдал** — один раз сразу предлагается дотест на грейд выше.
- **После присвоения** смена грейда вверх или вниз — не чаще раза в 90 дней (настраивается). Неудачная попытка повышения не понижает грейд; следующая — через 30 дней.
- Кандидат видит текущую категорию, историю опросов, тестов и присвоений, доступные попытки и дату, когда откроется смена грейда.

4.6. Как тест строится от вакансии

`GET /api/v1/catalog/test-plan` и `GET /api/v1/employer/needs/{id}/test-plan` показывают по описанию вакансии: целевой порог θ, темы, шаблоны, которые получат приоритет из-за стека вакансии, и примеры заданий для двух разных кандидатов. Например, для «Middle Go-разработчик: PostgreSQL, Kafka» приоритет получают `sql.group_by_having`, `sql.left_join_rows`, `go.defer_order`, `db.index_choice`, `db.isolation_anomaly`. Кандидаты А и Б получают одни и те же шаблоны и уровни, но разные запросы и данные — `WHERE o.amount > 600` у одного и `> 750` у другого.

5. Механика подбора и категоризация

5.1. Категоризация

Категория присваивается только по результату теста: сданный тест на заявленный грейд ⟶ категория «специализация · грейд». Самоописание и резюме влияют лишь на вторичные признаки (стек, ожидания) и никогда не определяют категорию. Кандидат без категории в поиске не участвует.

5.2. Сценарий выхода на контакт

1. Работодатель описывает потребность — формой или вставкой текста вакансии; система распознаёт специализацию, грейд, обязательные и желательные навыки, вилку, формат.
2. Система показывает **рекомендованные категории**: сколько кандидатов, сколько с ФСП, медианные ожидания, насколько категория покрывает стек.
3. **Подборка** — анонимные карточки: категория, сила результата теста, навыки с отметкой «подтверждён тестом / ФСП / заявлен», достижения ФСП, опыт (названия компаний — по желанию кандидата), ожидания. Без имени и контактов.
4. **Объяснение** у каждого кандидата: балл 0–100, разложение по признакам, причины («Все обязательные навыки: Go, PostgreSQL, Kafka; 1 подтверждён тестом», «Результат теста выше 71 % диапазона грейда Middle», «ФСП: Мастер спорта по продуктовому программированию, 2023») и пробелы («Нет обязательного навыка: Kubernetes», «Вместо PostgreSQL — близкий навык MySQL», «Навык Docker только заявлен»).

5. **Приглашение**: что предлагаем, сообщение, вилка «от–до» (обязательно), формат, способ связи с компанией; можно без вакансии.
6. Кандидат видит условия до общения и принимает или отклоняет с причиной (зарплата, стек, формат, компания, не ищу работу).
7. После принятия работодатель получает контакты; каждое обращение к контактам записывается в журнал.

5.3. Ранжирование

Жёсткие фильтры: кандидат видим и открыт к предложениям, категория нужной специализации, грейд из запрошенных (или соседний, если разрешено), формат и город совместимы.

Балл — взвешенная сумма признаков из [0; 1], веса подобраны на валидации (§6.3):

Признак	Вес	Расчёт
Обязательные навыки	0,41	Среднее по навыкам потребности: близость × доверие к источнику (тест / ФСП — 1,0; резюме — 0,7; заявлен — 0,5)
Грейд	0,15	1,0 — точное совпадение, 0,6 — на грейд выше, 0,5 — на грейд ниже
Результат теста	0,10	Положение θ внутри диапазона грейда
Достижения ФСП	0,10	ФСП-скор (§7.3)
Желательные навыки	0,08	Как обязательные
Зарплата	0,07	1,0, если ожидания в вилке; линейно до 0 при превышении на 30 %
Актуальность	0,05	exp(−дней с последней активности / 60): тест, задание, ответ на приглашение
Близость текстов	0,04	Полнотекстовая релевантность «о себе» описанию потребности (русская морфология)

Признаки без данных (нет желательных навыков, нет описания) выпадают, веса перенормируются. Без потребности — при просмотре категории — кандидаты упорядочены по силе профиля: 0,6 · тест + 0,25 · ФСП + 0,15 · актуальность.

Подбор 10 000 кандидатов с объяснением топ-20 занимает около 0,3 мс; в ранжирующем цикле нет аллокаций, тексты объяснений собираются только для итоговых кандидатов.

5.4. Уточнение без потери подборки

Подборка хранится. При уточнении фильтров она пересчитывается, но закреплённые кандидаты остаются (с пометкой, если не проходят новые фильтры), скрытые не возвращаются, а новые помечаются как «новые».

5.5. Чем это отличается от отклика на вакансию

- Инициатива у работодателя, который ищет по подтверждённым тестом категориям, а не по ключевым словам.
- Условия — в первом же сообщении: кандидат решает, тратить ли время.
- **Квота**: не более 30 приглашений компании за 7 дней и одно действующее приглашение на пару «компания–кандидат». Работодатель выбирает точечно, кандидат не тонет в спаме.
- Приглашение живёт 14 дней. Работодатель видит воронку и агрегированные причины отказов, кандидат — анонимную статистику спроса в своей категории: сколько компаний смотрели профиль, медианная вилка приглашений.
- Классический отклик остаётся дополнительным путём: контакты раскрываются и при собственном отклике.

6. Процедура валидации и результаты

Валидация — отдельный исполняемый модуль `crates/validate` (`make validate`), который вызывает **те же крейты**, что и API, и пишет отчёты в `docs/validation/`. Данных и разметки организаторы не дают, поэтому используется синтетика со скрытой «истиной».

6.1. Тестирование: точность и стабильность

Каждая сессия — настоящий движок; синтетические только ответы: вероятность верного ответа по 3PL при истинной $\theta \sim N(0; 1)$. 400 человек на специализацию, процедура как в API (заявленный грейд → при провале грейд ниже). Пересдача — вторая независимая процедура с другими заданиями.

Сценарий	Решение «выше/ниже порога» верно	Итоговый грейд = истинный	Отклонение ⩽ 1 грейда	Совпадение при пересдаче	κ Коэна	RMSE θ	Заданий
Честная заявка, точная калибровка	87,7 %	87,0 %	100 %	81,1 %	0,74	0,35	13,6
20 % завышают, 10 % занижают грейд	87,7 %	80,2 %	99,2 %	80,1 %	0,72	0,35	13,4
Ошибка калибровки заданий (a ×0,8…1,2, b ±0,3)	87,1 %	86,2 %	100 %	80,2 %	0,72	0,35	13,6

Ошибки только «вниз» и только у границы грейда: тест на заявленный грейд не повышает принудительно, а пограничный кандидат может пересдать на грейд выше. Неточная калибровка почти не ухудшает результат.

Дискриминативность: точечно-бисериальная корреляция верного ответа с истинной θ — в среднем 0,30, у 94 % шаблонов ⩾ 0,2. Самые слабые шаблоны выводятся в отчёт как кандидаты на перекалибровку. Низкая корреляция у шаблона с несколькими уровнями может быть артефактом смешения уровней: адаптивный тест показывает трудный уровень более сильным кандидатам, поэтому внутри уровня корреляция выше, чем по шаблону в целом (так было с `go.defer_order`: 0,30 / 0,21 / 0,50 по уровням при 0,13 в сумме; уровень перекалиброван).

6.2. Тестирование: устойчивость к утечке

Слабые кандидаты (истинная θ от −0,9 до −0,1, ниже порога Middle) сдают тест на Middle.

Утекло	Фиксированный тест: прошли	Наш тест: прошли
0 %	9,4 %	9,7 %
25 % ответов	65,2 %	9,6 %
50 % ответов	99,9 %	9,3 %
100 % ответов	100 %	10,4 %

Фиксированный тест — публичная форма из 15 самых информативных заданий. Для нашего теста «утечка» — база из всех экземпляров, показанных в 600 предыдущих сессиях на специализацию (~42 тыс. экземпляров); слабый кандидат отвечает верно, если ему выпал экземпляр из базы. Совпадения с базой случаются в 0,02–0,05 % заданий, поэтому утечка не даёт преимущества. Статистика person-fit ловит 11–12 % «сливщиков» фиксированного теста при 0,9 % ложных срабатываний.

6.3. Подбор: доля релевантных в топе

2 400 синтетических кандидатов, 120 потребностей для подбора весов и 120 отложенных для оценки. У кандидата есть скрытая истина (реальные навыки и θ) и наблюдаемый профиль с шумом: ~18 % заявленных навыков приукрашены, ~15 % реальных не заявлены, категория — из теста с ошибкой измерения, часть навыков подтверждена тестом. Релевантность (0–3) считается по истине функцией, отличной от скоринга. Релевантным считается оценка ⩾ 2.

Ранжирование	Precision@10	nDCG@10	MRR
Наш подбор (веса после валидации)	0,941 ± 0,017	0,835 ± 0,023	1,000
Наш подбор, исходные веса	0,888 ± 0,022	0,758 ± 0,027	0,986
Ключевые слова резюме + заявленный стаж	0,425 ± 0,030	0,356 ± 0,028	0,632
Случайный порядок в специализации	0,223 ± 0,025	0,173 ± 0,018	0,414

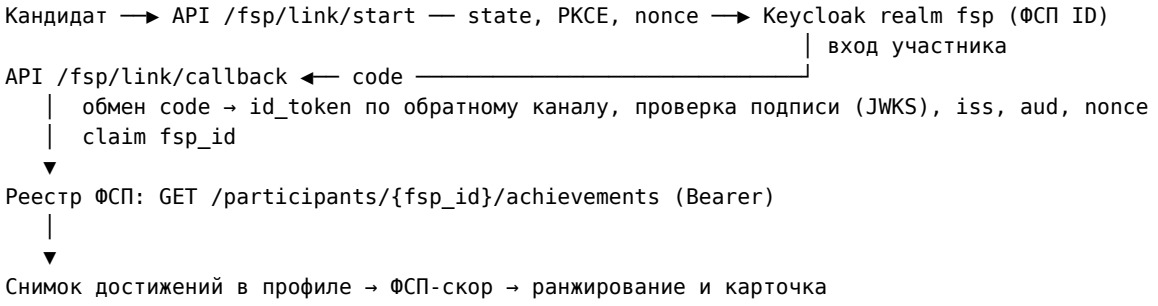
Абляции: без результата теста nDCG падает до 0,819, без графа близости навыков — до 0,829, только по навыкам — до 0,739. Исходные веса (обязательные навыки 0,30, тест 0,25) проигрывали; на трёх случайных сидах устойчиво лучшим оказался вариант с большим весом стека и грейда — он и принят.

6.4. Ограничения и экспертная проверка

Синтетическая «истина» задаётся командой, поэтому абсолютные цифры описывают поведение системы в заданных предпосылках, а не рынок. Сравнение с базовыми линиями и абляции от этого не зависят. Следующий шаг — экспертная разметка: 10–20 реальных вакансий партнёров ФСП, топ-20 выдачи каждой оценивают два эксперта по шкале 0–3, согласованность экспертов (κ) и Precision@10 считаются тем же модулем `validate`. Для тестов — пилот на участниках соревнований ФСП: сравнение грейда по тесту с разрядом и результатами соревнований, повторная сдача через неделю.

7. Интеграция с ФСП

7.1. Привязка ФСП ID



- ФСП ID работает на Keycloak; совместимость — стандартный OpenID Connect, клиент `fsp-talent` с обязательным PKCE S256. Claim `fsp_id` — атрибут пользователя Keycloak.
- Один ФСП ID можно привязать только к одному профилю. Отвязка удаляет снимок достижений.
- Если реестр недоступен, привязка сохраняется, синхронизация повторяется позже.
- Для интеграции с реальным реестром достаточно заменить адрес реестра и выдать API клиентские учётные данные Keycloak (client credentials или token exchange).

7.2. Какие данные получаем

Для каждого достижения: тип (разряд или соревнование), дисциплина (алгоритмическое, продуктовое программирование, программирование систем ИБ, систем ИИ, робототехники, БАС), разряд (юношеский, III–I, КМС, МС), уровень соревнования (региональное, межрегиональное, всероссийское, международное), место (1–3, финалист, участник), команда, роль, дата, ссылка на протокол.

7.3. ФСП-скор

Вклад достижения = вес уровня × вес места (или вес разряда) × релевантность дисциплины специализации × затухание с периодом полураспада 2 года. Итог — «шумное ИЛИ»: $1 - \prod(1 - v_i)$. Скор растёт с каждым достижением, но насыщается и не может перевесить тест и стек (вес в подборе — 0,10). Учитывается только высший разряд в дисциплине.

Кандидат без истории ФСП получает 0 без штрафа: в карточке нейтральное «Нет данных ФСП», отсутствие ФСП никогда не попадает в «пробелы». Кандидат может скрыть ФСП-данные — тогда они не участвуют и в ранжировании.

8. Безопасность и 152-ФЗ

8.1. Аутентификация

- Пароли — Argon2id (19 МиБ, t = 2, p = 1); при неизвестном email выполняется та же проверка, чтобы время ответа не раскрывало наличие аккаунта.
- Короткий access-токен (JWT EdDSA, 15 минут) и ротируемый refresh-токен (30 дней) в httpOnly-cookie; повторное использование старого refresh-токена отзывает всю цепочку. Выход заносит access-токен в deny-list.
- Защита от CSRF: изменяющие запросы с cookie принимаются только с собственного Origin.
- Ограничение частоты: вход, регистрация, повторные письма, приглашения, жалобы.

8.2. Персональные данные

- Email, ФИО и контакты кандидата хранятся зашифрованными (XChaCha20-Poly1305); поиск по email — через keyed-хеш (blind index).
- Версионированные согласия на обработку и публикацию с фиксацией времени, IP и User-Agent; отзыв согласия на публикацию скрывает профиль.
- Удаление аккаунта стирает персональные данные.
- Журнал доступа к контактам: кто, когда, на каком основании.
- Хранение — на сервере в РФ.

8.3. Разграничение доступа

- Роли кандидата и работодателя проверяются на уровне маршрутов; кандидат видит только свои приглашения, отклики и тесты, чужие ресурсы возвращают 404.
- **Контакты закрыты типами.** Карточка кандидата для работодателя не содержит полей контактов. Контакты можно получить только из «основания доступа», которое выдаёт единственная функция: она проверяет принятое приглашение или отклик и пишет журнал.

8.4. ГОСТ и сертифицированная криптография

Алгоритм хранится вместе с данными (строка хеша PHC, версия шифротекста), хеширование и шифрование — за интерфейсами. Переход на ГОСТ (PBKDF2-HMAC-Стрибог по Р 50.1.111-2016, «Кузнечик») и сертифицированное СКЗИ — это новая реализация интерфейса без изменения схемы данных; в MVP ГОСТ не используется, так как соответствие обеспечивает сертификация СКЗИ, а не сам алгоритм.

9. API

Все методы описаны в OpenAPI 3.1: спецификация `/api/v1/openapi.json`, Swagger UI — `/api/docs`. 81 операция на 68 путях.

- Версионирование: префикс `/api/v1`; несовместимые изменения — новая версия рядом.
- Формат: JSON, даты RFC 3339, деньги — целые рубли.
- Ошибки — RFC 9457 `application/problem+json` со стабильным машинным полем `code` (например, `grade_change_cooldown`, `invitation_quota_exceeded`, `contacts_not_granted`); 404 и для чужих ресурсов; 409 — конфликт состояния; 422 — ошибки полей; 429 — лимит.
- Аутентификация: httpOnly-cookie или заголовок `Authorization: Bearer`.

Группа	Что делает	Операций
`auth`	Регистрация с согласиями, подтверждение email, вход, обновление и завершение сессии	6
`account`	Текущий пользователь, согласия и их отзыв, удаление аккаунта, жалобы	5
`catalog`	Справочники, автодополнение навыков, документы согласий	3
`candidate`	Профиль, PDF-резюме и PDF-профиль, приватность, опрос, статистика спроса	9
`assessment`	Старт теста, текущее задание, ответ, история, категория, план теста под вакансию	8
`fsp`	Привязка ФСП ID (OIDC), dev-привязка, достижения, синхронизация, отвязка	6
`employer`	Профиль компании	2
`matching`	Потребности, распознавание текста вакансии, категории, подборка, поиск по банку, анонимный профиль	13
`invitations`	Приглашения с обеих сторон, квота и статистика, контакты	10
`vacancies`	Вакансии, публичный список, отклики	11
`tasks`	Регулярные задания	8

10. Вне MVP: концепции

Защита от фиктивных вакансий и недобросовестных работодателей. Уже работают: обязательная вилка, квота приглашений, срок жизни приглашения, жалобы. Следующий шаг: подтверждение корпоративного домена email и проверка ИНН/ОГРН по ЕГРЮЛ; рейтинг работодателя по доле ответов и жалобам; сравнение вилки с рынком категории (слишком высокая — подозрительно); поиск почти дублирующихся вакансий по эмбеддингам; ручная модерация по сигналам.

Интеграция с ATS. API-ключи компании с областями доступа; вебхуки на смену статусов приглашений и откликов; экспорт принятого кандидата в JSON Resume; коннекторы к популярным ATS через вебхуки и REST.

Масштабирование. API без состояния за балансировщиком; общие лимиты в Valkey; реплика PostgreSQL на чтение; эмбеддинги текстов (pgvector) — отдельным воркером; аналитика событий — выгрузка изменений в ClickHouse.

11. Библиотеки и компоненты

Весь код и зависимости — с открытыми лицензиями (MIT, Apache-2.0, BSD, ISC, MPL, OFL для шрифта).

Компонент	Версия	Назначение
Rust	1.85+ (сборка — 1.97)	Язык backend
axum / tokio	0.8.9 / 1.53	HTTP-сервер и асинхронный рантайм
tokio-postgres / deadpool-postgres	0.7.18 / 0.14.2	Драйвер и пул PostgreSQL
clorinde	2.0.0	Генерация типизированных запросов из SQL
refinery	0.10.0	Миграции
utoipa / utoipa-axum / utoipa-swagger-ui	6.0 / 0.3 / 10.0	OpenAPI и Swagger UI
argon2 / chacha20poly1305 / ed25519-dalek / jsonwebtoken	0.5.3 / 0.10.1 / 2.2 / 11.1	Криптография и токены
redis (для Valkey)	1.2	Лимиты и deny-list
lettre	0.11	Почта
reqwest	0.13	Запросы к ФСП ID и реестру
krilla / pdf-extract	0.8.2 / 0.12.1	Генерация и разбор PDF
metrics-exporter-prometheus / tracing	0.18 / 0.1	Метрики и логи
regex / rustc-hash / rand / rand_chacha	1.13 / 2.1 / 0.9 / 0.9	Разбор текста, словари, детерминированная генерация
Next.js / React / TypeScript	16.3 / 19.2 / 5	Frontend
TanStack Query / openapi-fetch / openapi-typescript	5 / 0.17 / 7	Данные и клиент из OpenAPI
Tailwind CSS / react-hook-form / zod	4 / 7 / 4	Стили и формы
PostgreSQL (+pgvector) / Valkey	17 / 8	Хранилище и кэш
Keycloak	26.7	ФСП ID (OIDC)
Caddy / Mailpit	2.11 / 1.31	Входной прокси с TLS, почта для демо
Prometheus / Loki / Alloy / Grafana	3.5 / 3.7 / 1.20 / 12.4	Наблюдаемость
JetBrains Mono	2.304	Фирменный шрифт ФСП

12. Сборка, развёртывание и запуск

12.1. Docker (рекомендуется)

Требования: Docker с Compose v2, 4 ГБ свободной памяти.

```
git clone https://github.com/K1NGSM4N3/fsp-hack && cd fsp-hack
cp .env.example .env
docker compose -f deploy/compose.yml --profile full up -d --build
```

Что	Адрес
Приложение	http://localhost:8080
Swagger UI	http://localhost:8080/api/docs
Письма подтверждения	http://localhost:8025
ФСП ID (Keycloak)	http://localhost:8081
Grafana	http://localhost:3001

При первом старте API применяет миграции и заполняет демо-данные: 6 вымышленных компаний, 12 вакансий, 240 кандидатов с симулированными тестами, 15 из них с ФСП.

Демо-доступы (пароль `demo12345`): работодатель `employer@demo.fsp`, кандидат `candidate@demo.fsp`, кандидаты `cand1@demo.fsp`…, участники ФСП ID в Keycloak `fsp-000001`…`fsp-000010`.

12.2. Локальная разработка без Docker

Требования: Rust ⩾ 1.85, Node.js ⩾ 20, PostgreSQL ⩾ 14. Либо `nix develop` — окружение со всеми инструментами.

```
make db       # база fsp_hack
make api      # API на :8000 (миграции при старте; ФСП ID — dev-привязкой)
make web      # интерфейс на :3000
make validate # отчёты валидации в docs/validation/
make test     # тесты
```

12.3. Для VPS

Задать в `.env` публичные адреса (`PUBLIC_URL`, `KC_HOSTNAME`, `FSP_OIDC_ISSUER`, `FSP_OIDC_REDIRECT_URL`), включить `COOKIE_SECURE=true`, сгенерировать новые ключи (`JWT_ED25519_SEED_B64`, `FIELD_ENC_KEY`, `BIDX_KEY` — по 32 байта в base64) сменить пароли Keycloak и Grafana и указать в `deploy/Caddyfile` домен вместо `:8080` — Caddy сам получит TLS-сертификат.